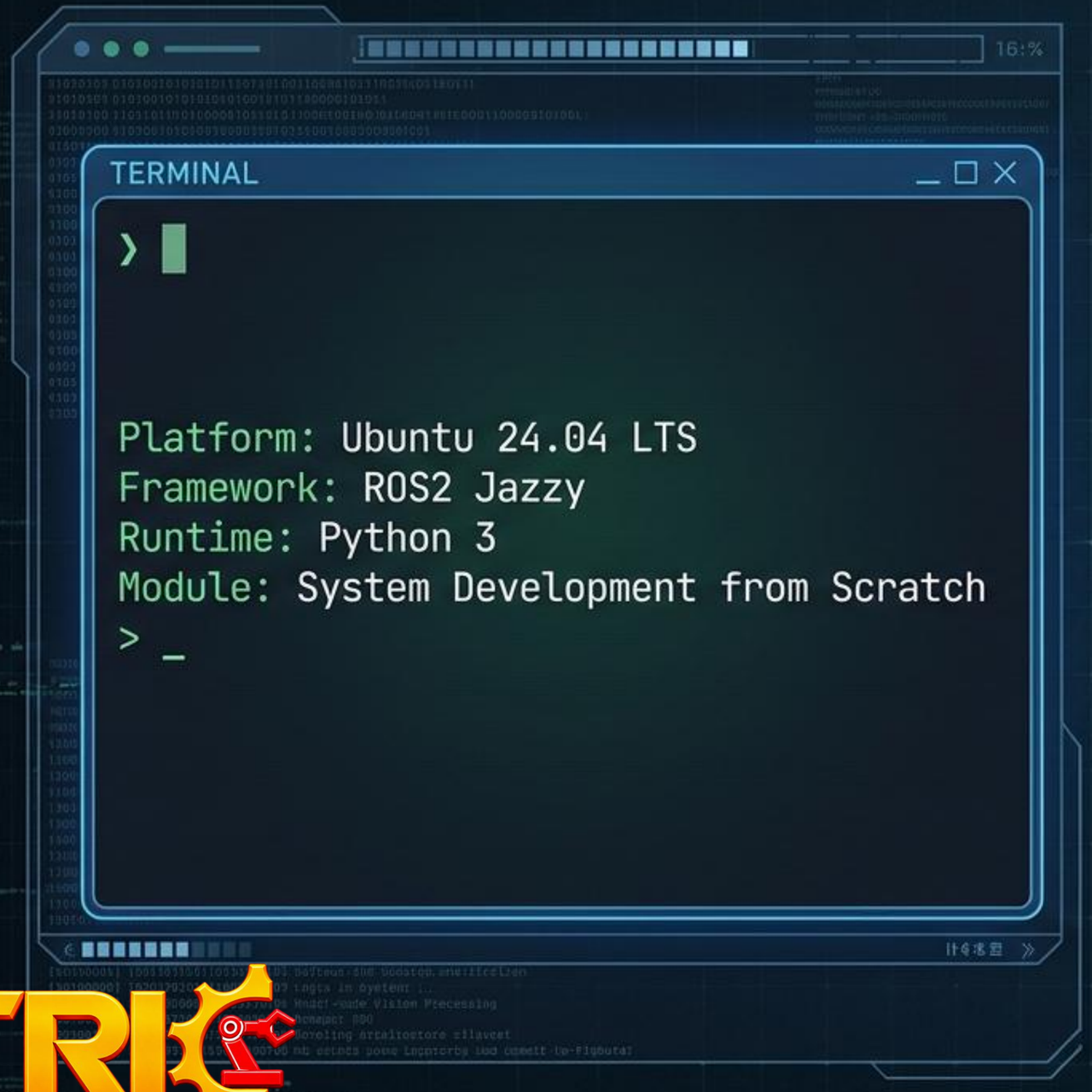


ROS2 Robotics System Integration

Multi-Node Vision Processing & System Architecture

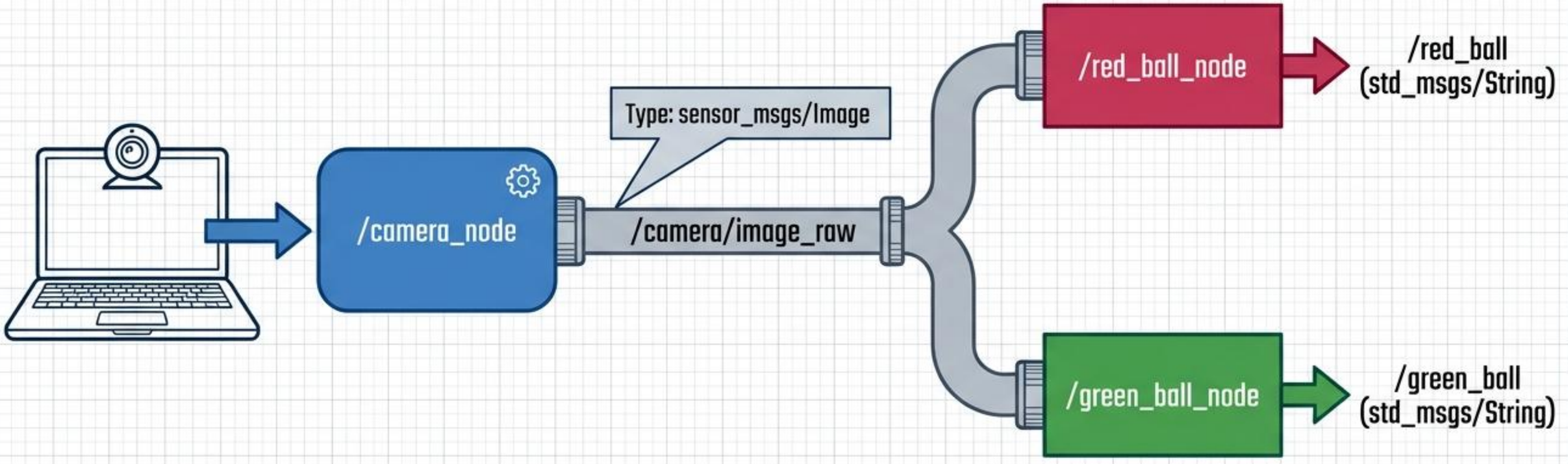
BOOTING: 87% COMPLETE

ROS2

TRIC
TECHNOLOGY OF ROBOTICS AND IOT CENTRE

Gemini Notebook

The System Blueprint: Decoupled Vision Architecture

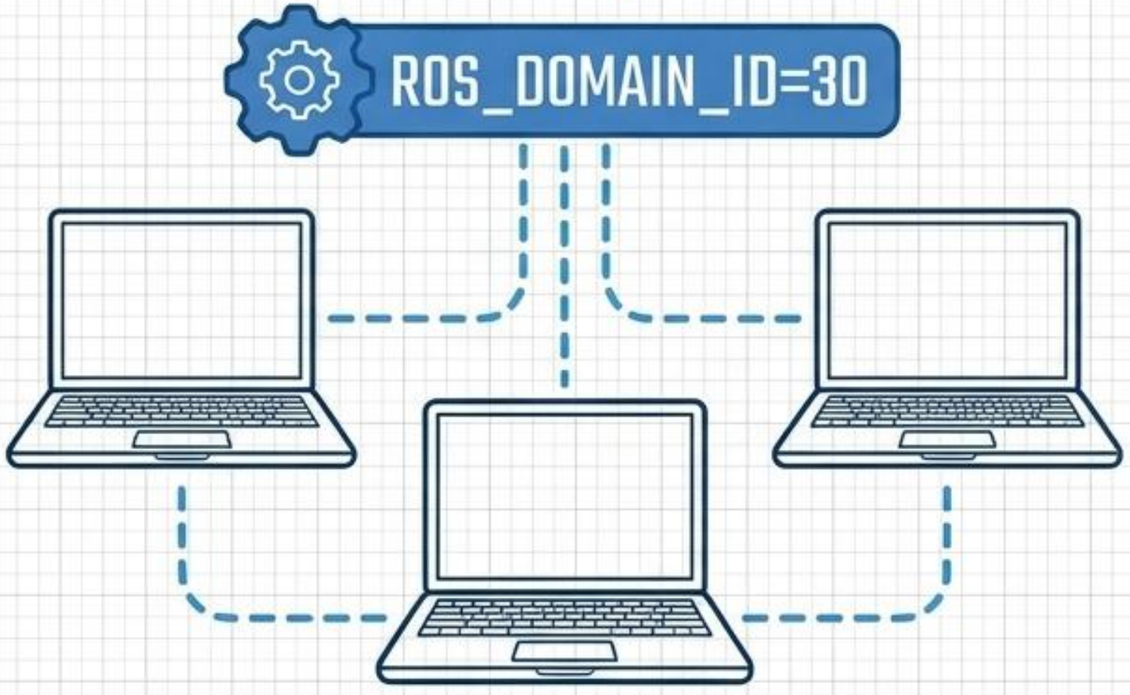


The **core advantage of ROS2**: The camera node does not know—or care—how many detection nodes are analyzing its frames.

Gemini Notebook

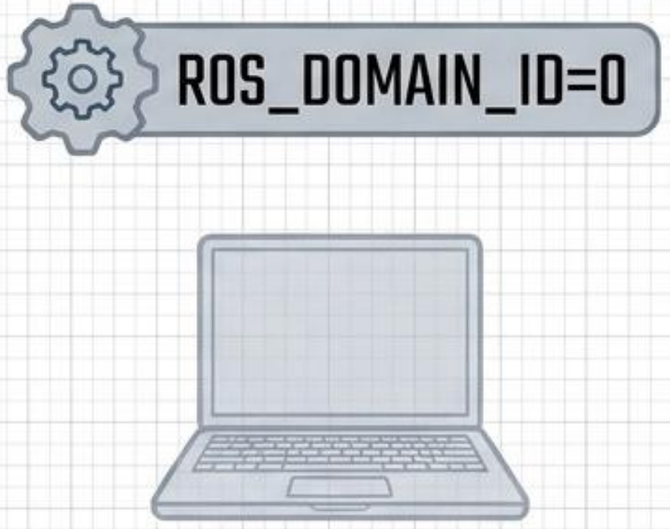
Network Isolation via DDS Discovery

The Workshop Subnet




```
$ export ROS_DOMAIN_ID=30
export ROS__AUTOMATIC_DISCOVERY_RANGE=SUBNET
```

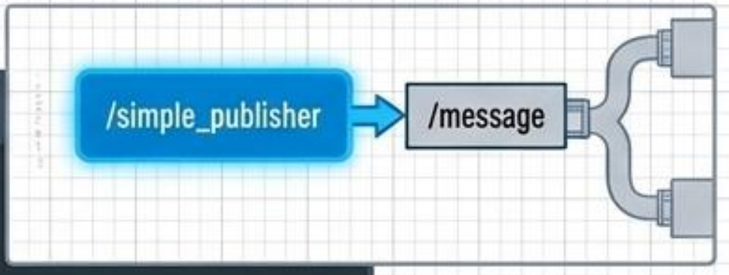
The Isolated Machine



DIAGNOSTIC NOTE: If a topic or node is missing, check `` `echo $ROS_DOMAIN_ID` `` in BOTH terminals. The workshop standard is 30. Mismatched IDs result in invisible nodes.

Gemini Notebook

Anatomy of a ROS2 Publisher (simple_publisher.py)




```python
class SimplePublisher(Node):
    def __init__(self):
        super().__init__('simple_publisher')
        self.publisher = self.create_publisher(String, '/message', 10)
        self.timer = self.create_timer(1.0, self.publish_message)

    def publish_message(self):
        msg = String()
        msg.data = f'Hello ROS2! Message {self.count}'
        self.publisher.publish(msg)
```

Defines topic name (/message), message type (String), and QoS queue size (10).

Injects the populated message payload directly into the ROS2 DDS space.

Non-blocking execution. Triggers the callback strictly at 1.0Hz intervals.

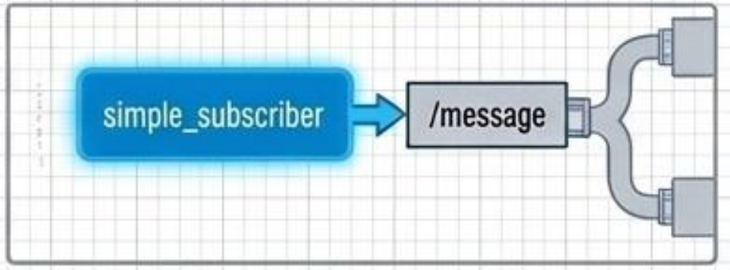
Gemini Notebook

Anatomy of a ROS2 Subscriber (simple_subscriber.py)

```python
class SimpleSubscriber(Node):
    def __init__(self):
        super().__init__('simple_subscriber')
        self.subscription = self.create_subscription(
            String, '/message', self.listener_callback, 10)

    def listener_callback(self, msg):
        self.get_logger().info(f'Received: {msg.data}')
```

Event-driven execution. Triggers only when a new message arrives on /message.



Publisher	Subscriber
Time-driven (create_timer)	Event-driven (listener_callback)
create_publisher	create_subscription
Requires no subscribers to run	Requires a matching topic to receive data

Gemini Notebook

Execution & Verification Flow

Phase 1: Build Workspace

```
cd ~/mindspark_ws
colcon build --symlink-install
source install/setup.bash
```

--symlink-install permits Python edits without rebuilding.

Phase 2: Execution (Dual Terminals)

```
ros2 run mindspark_robot simple_publisher
```

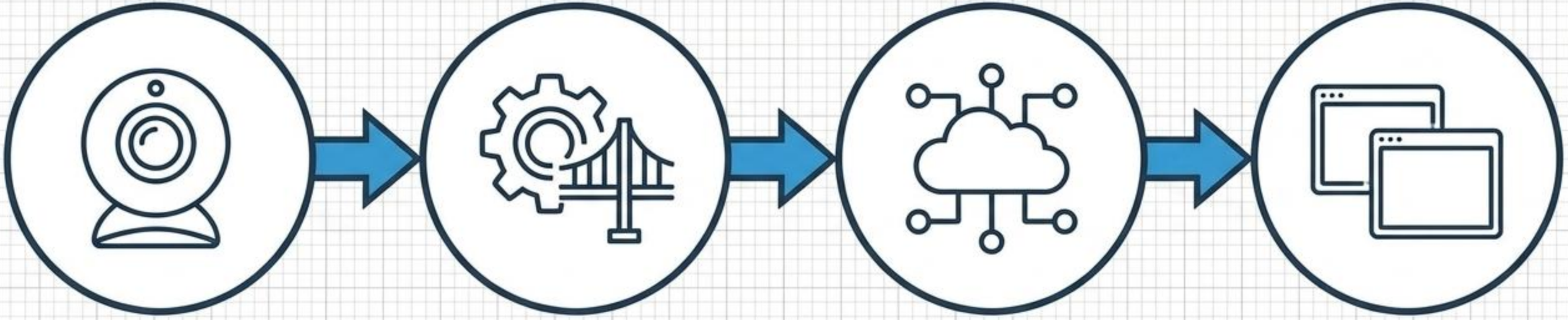
```
ros2 run mindspark_robot simple_subscriber
```

Phase 3: Inspection & Verification

```
ros2 topic info /message
-> Expect: Pub count 1, Sub count 1.
ros2 topic hz /message
-> Expect: ~1.0 Hz message rate.
```

Gemini Notebook

The Vision Data Pipeline



Hardware
(OpenCV VideoCapture)

cv_bridge

Translates OpenCV matrices (NumPy arrays) into ROS2 compatible formats.

sensor_msgs/Image

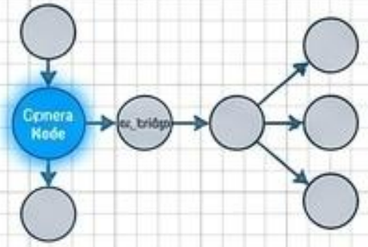
Encoding: bgr8. Standardized high--density payload for the ROS2 ecosystem.

Parallel CV Nodes

bridge.imgmsg_to_cv2 converts the network payload back to NumPy arrays for object detection.

Gemini Notebook

Camera Node Engineering: Payload Management




```
self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)

msg = self.bridge.cv2_to_imgmsg(frame,
    encoding='bgr8')
```

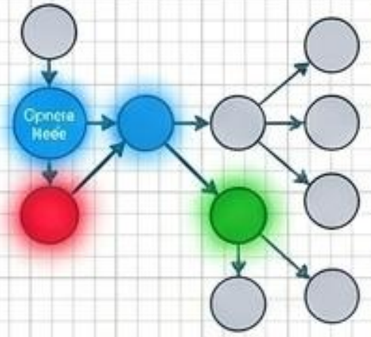
640x480 Resolution (Network Heavy)
640 * 480 * 3 channels (BGR8)
= 921,600 bytes/frame

320x240 Resolution
320 * 240 * 3 channels (BGR8)
= 230,400 bytes/frame

Downscaling to 320x240 reduces payload size by 75%, ensuring stable ~10Hz transmission across the DDS network without dropping frames.

Gemini Notebook

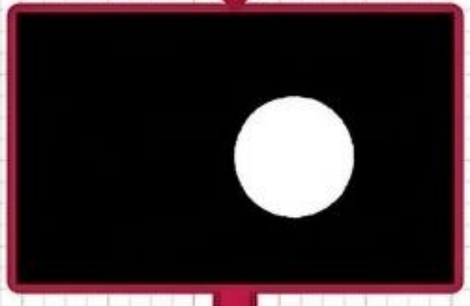
Parallel Processing: The OpenCV Masking Metaphor



sensor_msgs/Image (from /camera/image_raw)

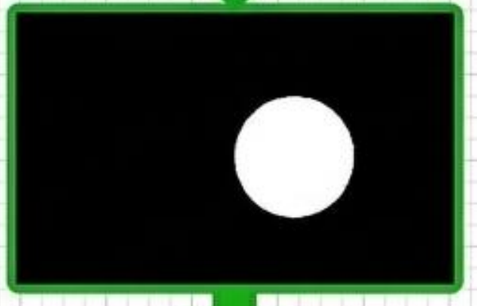


```
cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, [0,100,100] to [10,255,255])
```



Publishes String to **/red_ball**

```
cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, [35,70,70] to [85,255,255])
```

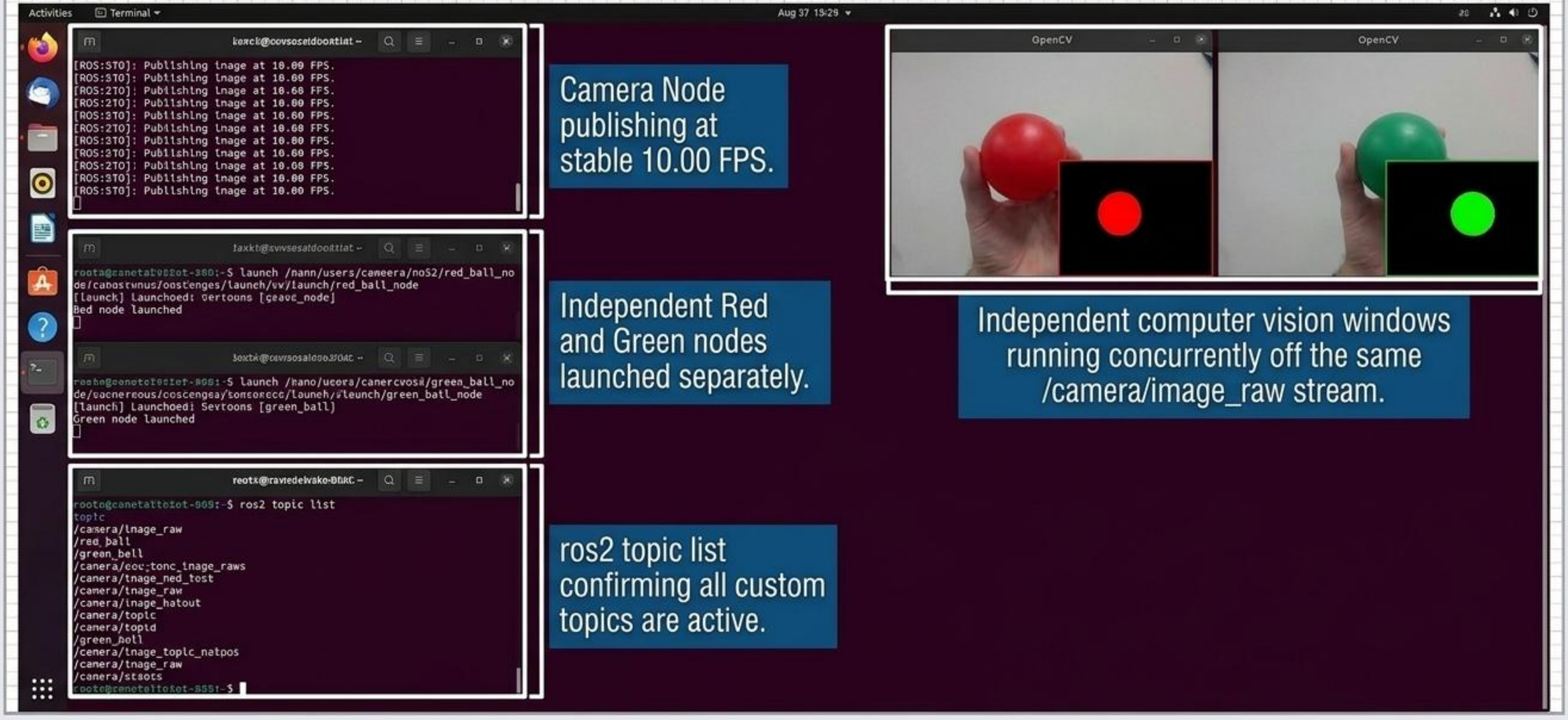


Publishes String to **/green_ball**

Gemini Notebook

Multi-Node Execution in Action

TRIC
TECHNOLOGY ROBOTICS AND IOT CENTRE



Camera Node publishing at stable 10.00 FPS.

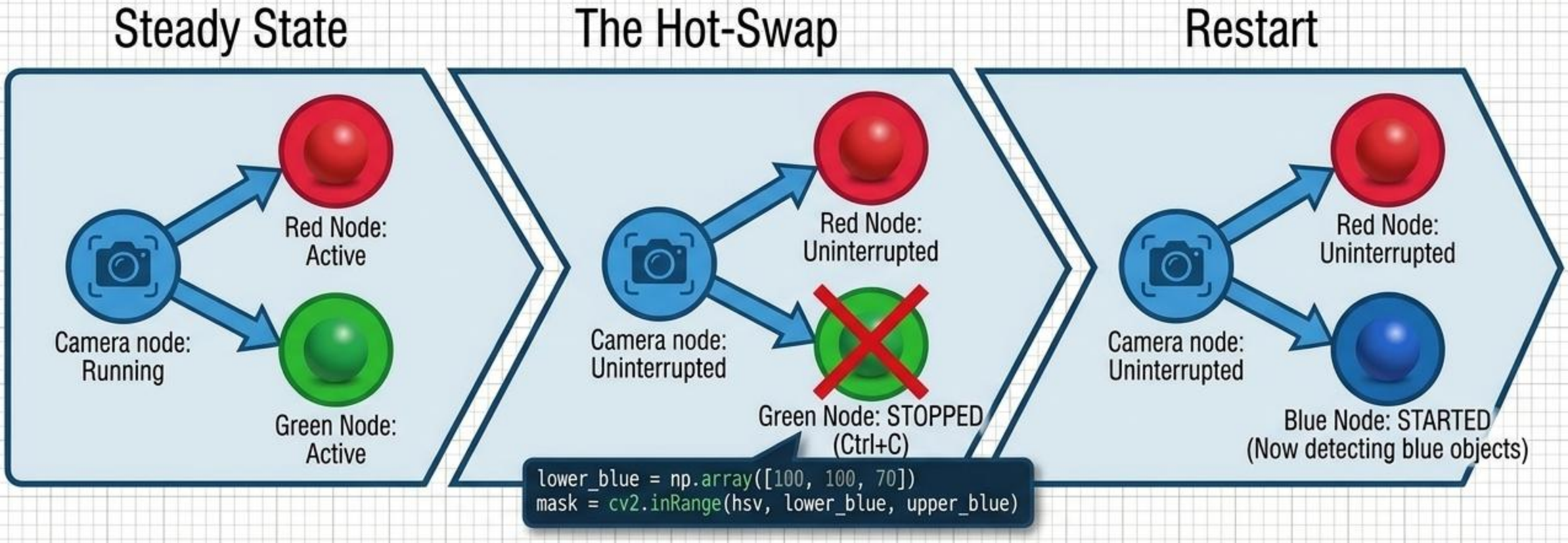
Independent Red and Green nodes launched separately.

ros2 topic list confirming all custom topics are active.

Independent computer vision windows running concurrently off the same /camera/image_raw stream.

Gemini Notebook

The “Aha!” Moment: Proving Node Independence



Because ROS2 decoupled the data (Pub) from the logic (Sub), we killed, rewrote, and restarted the green node while the red node and camera continued processing flawlessly.

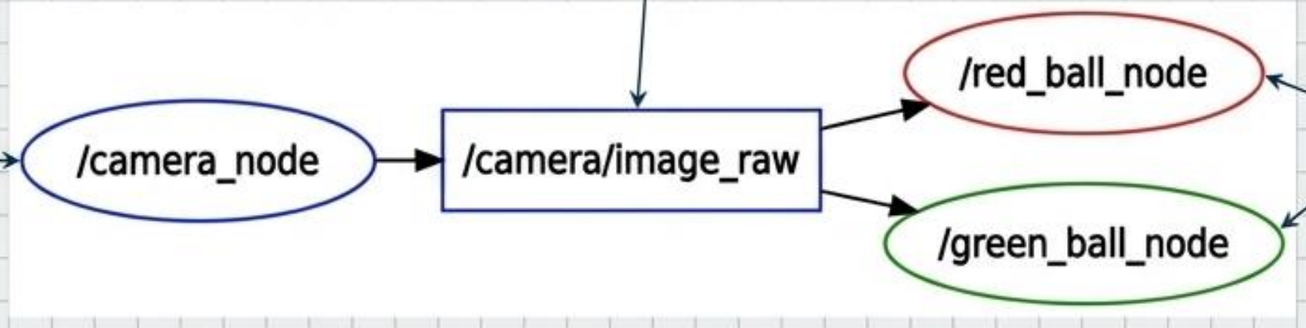
Gemini Notebook

Visualizing the Distributed System (rqt_graph)

TRI

The ROS2 Topic. Notice it is a rectangle, representing a data bus, not an active processing node.

The solitary data source. Requires exclusive lock on the /dev/video0 hardware.



The independent subscribers. If they disappear, ensure 'Hide -> Dead sinks' is unticked in the top menu.

Gemini Notebook

Diagnostic & Troubleshooting Matrix

Symptom Amber	Root Cause	Diagnostic CLI Command	Fix
Node/Topic missing	DDS Discovery Mismatch	echo $ROS_DOMAIN_ID	Match IDs, use ros2 daemon stop && start
ros2 run cannot find executable	setup.py not updated	cat setup.py	Add to console_scripts, colcon build, source install
Camera fails to open (Camera: False)	Hardware lock conflict	python3 -c 'import cv2...'	Close browser tabs; ensure no nested cv2.VideoCapture() in detector nodes
Low FPS or massive lag	BGR8 Payload too large	ros2 topic hz /camera/image_raw	Drop resolution to 320x240 in camera_node.py
Object visible but no bounding box	Bad lighting / HSV bounds	OpenCV display	Adjust lighting, tweak HSV arrays, adjust contour area > 200

Gemini Notebook

System Verification Checklist

TRI

- ✓ Basic Publisher/Subscriber verified via ros2 topic info
- ✓ Camera topic active streaming at 320x240
- ✓ Camera receiving rate confirmed at ~10 Hz
- ✓ Image topic successfully serving 2 simultaneous detection subscribers
- ✓ Hot-swap node independence proven (Green modified to Blue while Red active)
- ✓ rqt_graph matches the multi-node architecture blueprint

Terminal Verification

```
student@robot:~$ ros2 topic list -v
/camera/image_raw  [sensor_msgs/msg/Image] 2 subscribers
/red_ball/detection [std_msgs/msg/String] 1 subscriber
/green_ball/detection [std_msgs/msg/String] 1 subscriber
...
student@robot:~$ ros2 topic hz /camera/image_raw
student@robot:~$ ros2 topic hz  /camera/image_raw
average rate: 9.983 min: 0.100s max: 0.101s std dev: 0.00049s
 window: 10
...
[INFO] [red_ball_node]: Red Ball detected at (150, 120)
[INFO] [red_ball_node]: Geen Ball detected at (150, 120),
area: 3200
[INFO] [green_ball_node]: Green Ball detected at (200, 180),
area: 4500
[INFO] [hot_swap_node]: Modified green_ball_node to track
blue. Red active.
[INFO] [blue_ball_node]: Blue Ball detected at (200, 180),
area: 4500
```

POLITEKNIK MALAYSIA

Gemini Notebook

Engineering Knowledge Check

TRI

Why must ROS_DOMAIN_ID match in all terminals?

SSD discourtyk lellas on maichns domains to isolate swabneis: misnamicyed IGS cleare inuisible nodes sapou alqisinui aieaic SGI paycıewsiw :sıauqams aıeıosı oı suıewop suycıew uo sallal klanocsip

Difference between publishing FPS and ros2 topic hz?

‘klromten eht revo atar pevieɔaıy lat eht serusıam zy :peeps dool s,rapneıs eht sı SdF

Why can two detection nodes use the same /camera/image_raw topic?

‘uoitdmusuoɔ atap morf uoitarəneg atap seldnocad qnS/qnd ZSOZ

Why was 320x240 selected for the camera stream?

‘gnıkcaeneltoq klromtseu guituəvərd ‘08·x0-9 to pəlıadmoɔ % 75 ʎq əzis pɐoʃuɐd 8ɹƃ ɐɥʇ səɔuɐɹp ʇı

What was proven when the green node was stopped and modified?

‘mətsys ɔıjtoqir əritue eht tdursip tou seop apou guitıpdn ro guihsarc ənO ‘əɔnənɹedepul əpoN

POLITEKNIK MALAYSIA

Gemini Notebook